Text.Megaparsec: https://hackage.haskell.org/package/megaparsec
Official tutorial: https://markkarpov.com/tutorial/megaparsec.html#controlling-backtracking-with-try

Text.Megaparsec

```
type Parsec e s = ParsecT e s Identity
data ParsecT e s m a
                     -- e = error, s = stream

data State s e = State {
  stateInput :: s
  stateOffset :: !Int
  statePosState :: PosState s
  stateParseErrors :: [ParseError s e]
}

data PosState s = PosState {
  pstateInput :: s
  pstateOffset :: !Int
  pstateSourcePos :: !SourcePos
  pstateTabWidth :: Pos
  pstateLinePrefix :: String
}



-- Run
parse :: Parsec e s a -> String -> s -> Either (ParseErrorBundle s
e) a              -- runParser
parseMaybe :: (Ord e, Stream s) => Parsec e s a -> s -> Maybe a
parseTest ::
  (ShowErrorComponent e, Show a, VisualStream s, TraversableStream
s)
  => Parsec e s a -> s -> IO ()
runParser :: Parsec e s a -> String -> s -> Either
(ParseErrorBundle s e) a
runParser' :: Parsec e s a -> State s e -> (State s e, Either
(ParseErrorBundle s e) a)
runParserT :: Monad m
  => ParsecT e s m a -> String -> s -> m (Either (ParseErrorBundle
s e) a)
runParserT' :: Monad m
  => ParsecT e s m a -> State s e -> m (State s e, Either
(ParseErrorBundle s e) a)



-- Parser
class (Stream s, MonadPlus m) => MonadParsec e s m | m -> e s where
  parseError :: ParseError s e -> m a
  label :: String -> m a -> m a
  hidden :: m a -> m a
```

```
  try :: m a -> m a
  lookAhead :: m a -> m a
  notFollowedBy :: m a -> m ()
  withRecovery :: (ParseError s e -> m a) -> m a -> m a
  observing :: m a -> m (Either (ParseError s e) a)

  -- Token combinators
  eof :: m ()
  token :: (Token s -> Maybe a) -> Set (ErrorItem (Token s)) -> m a
  tokens :: (Tokens s -> Tokens s -> Bool) -> Tokens s -> m (Tokens
s)
  takeWhileP :: Maybe String -> (Token s -> Bool) -> m (Tokens s)
  takeWhile1P :: Maybe String -> (Token s -> Bool) -> m (Tokens s)
  takeP :: Maybe String -> Int -> m (Tokens s)

  -- State combinators
  getParserState :: m (State s e)
  updateParserState :: (State s e -> State s e) -> m ()
  mkParsec :: (State s e -> Reply e s a) -> m a


-- Token combinator
single :: MonadParsec e s m => Token s -> m (Token s)

satisfy :: MonadParsec e s m => (Token s -> Bool) -> m (Token s)
                           -- satisfy f = token f empty
anySingle :: MonadParsec e s m => m (Token s)
                           -- satisfy (const True)
anySingleBut :: MonadParsec e s m => Token s -> m (Token s)
                           -- anySingleBut t = satisfy (/= t)
oneOf :: (Foldable f, MonadParsec e s m) => f (Token s) -> m (Token
s)                     -- oneOf c = satisfy (`elem` c)
noneOf :: (Foldable f, MonadParsec e s m) => f (Token s) -> m
(Token s)                         -- noneOf c = satisfy (`notElem` c)
chunk :: MonadParsec e s m => Tokens s -> m (Tokens s)
                           -- tokens (==)
(<?>) :: MonadParsec e s m => m a -> String -> m a
                           -- label
match :: MonadParsec e s m => m a -> m (Tokens s, a)
takeRest :: MonadParsec e s m => m (Tokens s)
                           -- takeWhileP Nothing (const True)

-- State combinator
atEnd :: MonadParsec e s m => m Bool
getInput :: MonadParsec e s m => m s
setInput :: MonadParsec e s m => s -> m ()
getSourcePos :: (TraversableStream s, MonadParsec e s m) => m
SourcePos
getOffset :: MonadParsec e s m => m Int
setOffset :: MonadParsec e s m => Int -> m ()
setParserState :: MonadParsec e s m => State s e -> m ()

-- Error combinator
```

```
failure :: MonadParsec e s m
                          -- TrivialError
  => Maybe (ErrorItem (Token s)) -> Set (ErrorItem (Token s)) -> m
a
fancyFailure :: MonadParsec e s m => Set (ErrorFancy e) -> m a
                          -- FancyError
unexpected :: MonadParsec e s m => ErrorItem (Token s) -> m a
                          -- TrivialError
customFailure :: MonadParsec e s m => e -> m a
                          -- FancyError + ErrorCustom

region :: MonadParsec e s m => (ParseError s e -> ParseError s e)
-> m a -> m a
registerParseError :: MonadParsec e s m => ParseError s e -> m ()
registerFailure :: MonadParsec e s m
  => Maybe (ErrorItem (Token s)) -> Set (ErrorItem (Token s)) -> m
()
registerFancyFailure :: MonadParsec e s m => Set (ErrorFancy e) ->
m ()
```

Text.Megaparsec.Char

```
char :: (MonadParsec e s m, Token s ~ Char) => Token s -> m (Token
s)
char' :: (MonadParsec e s m, Token s ~ Char) => Token s -> m (Token
s)
string :: MonadParsec e s m => Tokens s -> m (Tokens s)
                       -- tokens (==)
string' :: (MonadParsec e s m, FoldCase (Tokens s)) => Tokens s ->
m (Tokens s)
charCategory :: (MonadParsec e s m, Token s ~ Char) =>
GeneralCategory -> m (Token s)
categoryName :: GeneralCategory -> String



--
newline :: (MonadParsec e s m, Token s ~ Char) => m (Token s)
crlf :: forall e s m. (MonadParsec e s m, Token s ~ Char) => m
(Tokens s)
eol :: forall e s m. (MonadParsec e s m, Token s ~ Char) => m
(Tokens s)
tab :: (MonadParsec e s m, Token s ~ Char) => m (Token s)
space :: (MonadParsec e s m, Token s ~ Char) => m ()
                       -- Data.Char.isSpace i.e. ' ' \t \r \n \v \f
hspace :: (MonadParsec e s m, Token s ~ Char) => m ()
space1 :: (MonadParsec e s m, Token s ~ Char) => m ()
hspace1 :: (MonadParsec e s m, Token s ~ Char) => m ()
controlChar :: (MonadParsec e s m, Token s ~ Char) => m (Token s)
spaceChar :: (MonadParsec e s m, Token s ~ Char) => m (Token s)

-- Alphanum
```

```
upperChar :: (MonadParsec e s m, Token s ~ Char) => m (Token s)
lowerChar :: (MonadParsec e s m, Token s ~ Char) => m (Token s)
letterChar :: (MonadParsec e s m, Token s ~ Char) => m (Token s)
alphaNumChar :: (MonadParsec e s m, Token s ~ Char) => m (Token s)
digitChar :: (MonadParsec e s m, Token s ~ Char) => m (Token s)
binDigitChar :: (MonadParsec e s m, Token s ~ Char) => m (Token s)
octDigitChar :: (MonadParsec e s m, Token s ~ Char) => m (Token s)
hexDigitChar :: (MonadParsec e s m, Token s ~ Char) => m (Token s)
numberChar :: (MonadParsec e s m, Token s ~ Char) => m (Token s)

-- Symbols
markChar :: (MonadParsec e s m, Token s ~ Char) => m (Token s)
printChar :: (MonadParsec e s m, Token s ~ Char) => m (Token s)
punctuationChar :: (MonadParsec e s m, Token s ~ Char) => m (Token
s)
symbolChar :: (MonadParsec e s m, Token s ~ Char) => m (Token s)
separatorChar :: (MonadParsec e s m, Token s ~ Char) => m (Token s)
asciiChar :: (MonadParsec e s m, Token s ~ Char) => m (Token s)
latin1Char :: (MonadParsec e s m, Token s ~ Char) => m (Token s)
```

Text.Megaparse.Char.Lex

```
charLiteral :: (MonadParsec e s m, Token s ~ Char) => m Char
decimal :: (MonadParsec e s m, Token s ~ Char, Num a) => m a
binary :: forall e s m a. (MonadParsec e s m, Token s ~ Char, Num
a) => m a
octal :: forall e s m a. (MonadParsec e s m, Token s ~ Char, Num a)
=> m a
hexadecimal :: forall e s m a. (MonadParsec e s m, Token s ~ Char,
Num a) => m a
scientific :: forall e s m. (MonadParsec e s m, Token s ~ Char) =>
m Scientific
float :: (MonadParsec e s m, Token s ~ Char, RealFloat a) => m a
signed :: (MonadParsec e s m, Token s ~ Char, Num a) => m () -> m a
-> m a

-- Whitespace + comments
space :: MonadParsec e s m
  => m ()
                          -- whitespace
  -> m ()
                          -- line comment
  -> m ()
                          -- block comment
  -> m ()
lexeme :: MonadParsec e s m => m () -> m a -> m a
symbol :: MonadParsec e s m => m () -> Tokens s -> m (Tokens s)
symbol' :: (MonadParsec e s m, FoldCase (Tokens s)) => m () ->
Tokens s -> m (Tokens s)
skipLineComment :: (MonadParsec e s m, Token s ~ Char) => Tokens s
-> m ()
```

```
skipBlockComment :: (MonadParsec e s m, Token s ~ Char) => Tokens s
-> Tokens s -> m ()
skipBlockCommentNested :: (MonadParsec e s m, Token s ~ Char) =>
Tokens s -> Tokens s -> m ()

-- Indents
indentLevel :: (TraversableStream s, MonadParsec e s m) => m Pos
incorrectIndent :: MonadParsec e s m => Ordering -> Pos -> Pos -> m
a
indentGuard :: (TraversableStream s, MonadParsec e s m) => m () ->
Ordering -> Pos -> m Pos
nonIndented :: (TraversableStream s, MonadParsec e s m) => m () ->
m a -> m a
data IndentOpt m a b
  = IndentNone a
  | IndentMany (Maybe Pos) ([b] -> m a) (m b)
  | IndentSome (Maybe Pos) ([b] -> m a) (m b)
indentBlock :: (TraversableStream s, MonadParsec e s m, Token s ~
Char) => m () -> m (IndentOpt m a b) -> m a
lineFold :: (TraversableStream s, MonadParsec e s m) => m () -> (m
() -> m a) -> m a
```

Control.Monad.Combinators

```
-- Alternative
(<|>) :: Alternative f => f a -> f a -> f a
optional :: Alternative f => f a -> f (Maybe a)
many :: MonadPlus m => m a -> m [a]
some :: MonadPlus m => m a -> m [a]

empty :: Alternative f => f a
between :: Applicative m => m open -> m close -> m a -> m a
choice :: (Foldable f, Alternative m) => f (m a) -> m a
count :: Monad m => Int -> m a -> m [a]
count' :: MonadPlus m => Int -> Int -> m a -> m [a]
eitherP :: Alternative m => m a -> m b -> m (Either a b)
endBy :: MonadPlus m => m a -> m sep -> m [a]
endBy1 :: MonadPlus m => m a -> m sep -> m [a]
manyTill :: MonadPlus m => m a -> m end -> m [a]
manyTill_ :: MonadPlus m => m a -> m end -> m ([a], end)
someTill :: MonadPlus m => m a -> m end -> m [a]
someTill_ :: MonadPlus m => m a -> m end -> m ([a], end)
option :: Alternative m => a -> m a -> m a
sepBy :: MonadPlus m => m a -> m sep -> m [a]
sepBy1 :: MonadPlus m => m a -> m sep -> m [a]
sepEndBy :: MonadPlus m => m a -> m sep -> m [a]
sepEndBy1 :: MonadPlus m => m a -> m sep -> m [a]
skipMany :: MonadPlus m => m a -> m ()
skipSome :: MonadPlus m => m a -> m ()
skipCount :: Monad m => Int -> m a -> m ()
```

```
skipManyTill :: MonadPlus m => m a -> m end -> m end
skipSomeTill :: MonadPlus m => m a -> m end -> m end
```

Text.Megaparsec.Pos

```
data Pos
mkPos :: Int -> Pos
unPos :: Pos -> Int
pos1 :: Pos                                                    --
mkPos 1
defaultTabWidth :: Pos                                         --
mkPos 8

newtype InvalidPosException = InvalidPosException Int

data SourcePos = SourcePos {
  sourceName :: FilePath
  sourceLine :: !Pos
  sourceColumn :: !Pos
}
initialPos :: FilePath -> SourcePos
sourcePosPretty :: SourcePos -> String
```

Megaparsec
Features
- **backtracking**: **non-atomic** combinators do NOT backtrack i.e. does not **reset** consumed tokens if the parser fails.
More precisely, a different **path** is NOT tried if at least one **atomic parser** consumes
Need to explicitly use **try combinator** to backtrack

p = (char 'a' >> char 'b') <|> (char 'a' >> char 'c')	BAD
parseTest p "ab"	success
parseTest p "ac"	fail.
	option 2 (char 'a' >> char 'c') is not even tried because option 1 (char 'a' >> char 'b') consumed 'a'
p = try (char 'a' >> char 'b') <|> (char 'a' >> char 'c')	GOOD. Need **try** to backtrack
parseTest p "ab"	success
parseTest p "ac"	success
p = (char 'a >> char 'b') <|> (char 'd' >> char 'c')	GOOD. No need for try.
parseTest p "ab"	success
parseTest p "dc"	success

Atomic combinators:

token ...	atomic (backtracks)
tokens ...	atomic (backtracks)
char	atomic. Uses token under the hood.
string	atomic. Uses tokens under the hood.
takeWhileP ...	atomic (backtracks)

takeWhile1 ...	atomic (backtracks)
token ... >> token ...	non-atomic
try (token ... >> token ...)	backtrack with try
p <|> q	non-atomic if p or q is non-atomic
many p	non-atomic if p is non-atomic

e.g. non-atomic

```
ghci> import Text.Megaparsec
ghci> import Text.Megaparsec.Char
ghci> import Data.Text (Text)
ghci> import Data.Void
ghci> type Parser a = Parsec Void Text

-- e.g. atomic
-- `char` is a wrapper for `token`, hence atomic
ghci> p = (char 'a') <|> (char 'b') :: Parser Char
                  -- OK
ghci> parseTest p "aaa"
                  -- option 1
'a'
ghci> parseTest p "bbb"
                  -- option 2
'b'
ghci> parseTest p "c"
                  -- neither
1:1:
  |
1 | c
  | ^
unexpected 'c'
expecting 'a' or 'b'


-- e.g. non-atomic
-- (>>) is non-atomic
ghci> p = (char 'a' >> char 'b') <|> (char 'd' >> char 'c') ::
Parser Char           -- OK. No need for backtrack if the first
terms are different.
ghci> parseTest p "ab"
                      -- option 1
'b'
ghci> parseTest p "dc"
                      -- option 2
'c'
ghci> parseTest p "e"
                      -- neither
1:1:
  |
1 | e
  | ^
unexpected 'e'
expecting 'a' or 'd'
```

```
ghci> p = (char 'a' >> char 'b') <|> (char 'a' >> char 'c') ::
Parser Char          -- BAD. Need backtrack if multiple paths
share the same first term
ghci> parseTest p "ab"
                     -- option 1
'b'
ghci> parseTest p "ac"
                     -- BAD. option 2 fails
1:2:
  |
1 | ac
  |  ^
unexpected 'c'
expecting 'b'

ghci> p = try (char 'a' >> char 'b') <|> (char 'a' >> char 'c') ::
Parser Char      -- OK. Explicit backtrack with try
ghci> parseTest p "ab"
                     -- option 1
'b'
ghci> parseTest p "ac"
                     -- option 2
'c'
ghci> parseTest p "d"
                     -- neither
1:1:
  |
1 | d
  | ^
unexpected 'd'
expecting 'a'



-- e.g. non-atomic with many
ghci> p = many (char 'a' >> char 'b') :: Parser String
                  -- BAD. Fails if last term consumes 'a'
ghci> parseTest p "ababc"

"bb"
ghci> parseTest p "abac"
                  -- Bad error
1:4:
  |
1 | abac
  |    ^
unexpected 'c'
expecting 'b'

ghci> p = many (try $ char 'a' >> char 'b') :: Parser String
                  -- OK. Explicit backtrack with try
ghci> parseTest p "ababc"
"bb"
ghci> parseTest p "abac"
```

```
"b"
```

e.g. atomic

```
-- e.g. string is based on atomic `tokens`
ghci> p = string "ab" <|> string "ac" :: Parser Text        -- OK
ghci> parseTest p "ab"                                      --
option 1
"ab"
ghci> parseTest p "ac"                                      --
option 2
"ac"
ghci> parseTest p "ad"                                      --
neither
1:1:
  |
1 | ad
  | ^^
unexpected "ad"
expecting "ab" or "ac"

-- e.g. `takeWhile1P` is atomic
ghci> p = takeWhile1P Nothing (=='a') <|> takeWhile1P Nothing
(=='b') :: Parser Text
ghci> parseTest p "aac"                                     --
option 1
"aa"
ghci> parseTest p "bbc"                                     --
option 2
"bb"
ghci> parseTest p "ddd"                                     --
neither
1:1:
  |
1 | ddd
  | ^
unexpected 'd'
```

States
State represents current state of parser i.e. unconsumed stream + # consumed + registered errors
PosState represents current location

```
  data State s e = State {
    stateInput :: s                    -- unconsumed stream
    stateOffset :: !Int                -- number of stream Tokens consumed
so far
                                       -- i.e. size of stateInput + stateOffset =
total number of Tokens
    statePosState :: PosState s        -- for computing error position. See
TraverseStream below.
```

```
  stateParseErrors :: [ParseError s e]
 }
```

e.g.

```
ghci> import Text.Megaparsec
ghci> Import Data.Void
ghci> type Parser = Parsec Void String

ghci> d :: Parser (); d = do state <- getParserState ; trace (show
state) $ return ()

-- e.g. single token
ghci> parseTest (d >> (single 'a') >> d) "abc"
State {stateInput = "abc", stateOffset = 0, statePosState =
PosState {pstateInput = "abc", pstateOffset = 0, pstateSourcePos =
SourcePos {sourceName = "", sourceLine = Pos 1, sourceColumn = Pos
1}, pstateTabWidth = Pos 8, pstateLinePrefix = ""},
stateParseErrors = []}
State {stateInput = "bc", stateOffset = 1, statePosState = PosState
{pstateInput = "abc", pstateOffset = 0, pstateSourcePos = SourcePos
{sourceName = "", sourceLine = Pos 1, sourceColumn = Pos 1},
pstateTabWidth = Pos 8, pstateLinePrefix = ""}, stateParseErrors =
[]}

-- e.g. multiple tokens
ghci> parseTest (d >> (string "abc") >> d) "abcde"
State {stateInput = "abcde", stateOffset = 0, statePosState =
PosState {pstateInput = "abcde", pstateOffset = 0, pstateSourcePos
= SourcePos {sourceName = "", sourceLine = Pos 1, sourceColumn =
Pos 1}, pstateTabWidth = Pos 8, pstateLinePrefix = ""},
stateParseErrors = []}
State {stateInput = "de", stateOffset = 3, statePosState = PosState
{pstateInput = "abcde", pstateOffset = 0, pstateSourcePos =
SourcePos {sourceName = "", sourceLine = Pos 1, sourceColumn = Pos
1}, pstateTabWidth= Pos 8, pstateLinePrefix = ""}, stateParseErrors
= []}

-- e.g. multiple lines
ghci> parseTest (d >> string "abc" >> space >> string "def" >> d)
"abc\ndefzz"
State {stateInput = "abc\ndefzz", stateOffset = 0, statePosState =
PosState {pstateInput = "abc\ndefzz", pstateOffset = 0,
pstateSourcePos = SourcePos {sourceName = "", sourceLine = Pos 1,
sourceColumn = Pos 1}, pstateTabWidth = Pos 8, pstateLinePrefix =
""}, stateParseErrors = []}
State {stateInput = "zz", stateOffset = 7, statePosState = PosState
{pstateInput = "abc\ndefzz", pstateOffset = 0, pstateSourcePos =
SourcePos {sourceName = "", sourceLine = Pos 1, sourceColumn = Pos
1}, pstateTabWidth = Pos 8, pstateLinePrefix = ""},
stateParseErrors = []}
```

Text.Megaparsec.Error

```
data ParseErrorBundle s e = ParseErrorBundle {
  bundleErrors :: NonEmpty (ParseError s e)
  bundlePosState :: PosState s
}

data ParseError s e
  = TrivialError Int (Maybe (ErrorItem (Token s))) (Set (ErrorItem
(Token s)))               -- TrivialError <offset> <unexpected
value> <expected values>
  | FancyError Int (Set (ErrorFancy e))

data ErrorItem t
  = Tokens (NonEmpty t)
  | Label (NonEmpty Char)
  | EndOfInput

data ErrorFancy e
  = ErrorFail String
  | ErrorIndentation Ordering Pos Pos
  | ErrorCustom e

mapParseError :: Ord e' => (e -> e') -> ParseError s e ->
ParseError s e'
errorOffset :: ParseError s e -> Int
setErrorOffset :: Int -> ParseError s e -> ParseError s e
attachSourcePos :: (Traversable t, TraversableStream s)
  => (a -> Int) -> t a -> PosState s -> (t (a, SourcePos), PosState
s)

-- Pretty print
class Ord a => ShowErrorComponent a where
 showErrorComponent :: a -> String
 errorComponentLen :: a -> Int
instance ShowErrorComponent Void

errorBundlePretty :: forall s e. (VisualStream s, TraversableStream
s, ShowErrorComponent e)
  => ParseErrorBundle s e -> String
parseErrorPretty :: (VisualStream s, ShowErrorComponent e) =>
ParseError s e -> String
parseErrorTextPretty :: forall s e. (VisualStream s,
ShowErrorComponent e)
  => ParseError s e -> String
showErrorItem :: VisualStream s => Proxy s -> ErrorItem (Token s)
-> String
```

```
import Data.Set qualified as S
import Data.List.NonEmpty qualified as N

do
  o <- getOffset
  let unexpectedItem = Tokens $ N.singleton "a"
      expectedItem = Tokens $ N.singleton "b"

  parseError $
    TrivialError o (Just unexpectedItem) (S.singleton expectedItem)
  failure (Just unexpectedItem) (S.singleton expectedItem)
    -- same

  parseError $ FancyError o $ ErrorFail "OOPS"
  fancyError $ ErrorFail "OOPS"
    -- same
```

==========

Stream

Text.Megaparsec.Stream

```
class (Ord (Token s), Ord (Tokens s)) => Stream s where
  type Token s :: Type
  type Tokens s :: Type
  tokenToChunk :: Proxy s -> Token s -> Tokens s
  tokensToChunk :: Proxy s -> [Token s] -> Tokens s
  chunkToTokens :: Proxy s -> Tokens s -> [Token s]
  chunkLength :: Proxy s -> Tokens s -> Int
  chunkEmpty :: Proxy s -> Tokens s -> Bool
  take1_ :: s -> Maybe (Token s, s)
  takeN_ :: Int -> s -> Maybe (Tokens s, s)
  takeWhile_ :: (Token s -> Bool) -> s -> (Tokens s, s)

class Stream s => VisualStream s where
  showTokens :: Proxy s -> NonEmpty (Token s) -> String
  tokensLength :: Proxy s -> NonEmpty (Token s) -> Int

class Stream s => TraversableStream s where
  reachOffset :: Int -> PosState s -> (Maybe String, PosState s)
  reachOffsetNoLine :: Int -> PosState s -> PosState s

-- Internal
newtype ShareInput a = ShareInput {
  unShareInput :: a
}
newtype NoShareInput a = NoShareInput {
  unNoShareInput :: a
}
```

Stream
Default streams are

Stream ByteString	Token ByteString = Word8	strict ByteString	i.e. Data.ByteString
Stream ByteString	Token ByteString = Word8	lazy ByteString	i.e. Data.ByteString.Lazy
Stream Text	Token ByteString = Char	strict Text	i.e. Data.Text (strict)
Stream Text	Token ByteString = Char	lazy Text	i.e. Data.Text.Lazy
Stream [a]	Token ByteString = a		

VisualStream
Display tokens of a stream.
Used to **highlight token** in error message.
e.g.

```
1 | ABDEF              tokens [A]          showTokens = "A", tokensLength = 1
  | ^
1 | ABDEF              tokens [A,B,D,E]  showTokens = "ABDE", tokensLength = 4
  | ^^^^
1 | HelloA             tokens [Hello]      showTokens = "Hello", tokenLength = 5
  | ^^^^
```

TraverseStream
Used to compute **error location** i.e. **line/column number** of an error

```
 -- TraverseStream.reachOffset
 -- e.g.
 --       tokens  ['1','2','3','4','a','b','c','d','\n','e','f','g']
 --       consumed ['1','2','3','4']
 --
 reachOffset :: Int -> PosState s -> (Maybe String, PosState s)
 reachOffset
   offset = ...,                                    number of tokens consumed in current state
   PosState {
     pstateOffset = 4,                              number of tokens consumed in previous state
     pstateInput = ['a','b','c','d','\n','e','f','g'],   unconsumed tokens
     pstateSourcePos = SourcePos {
       sourceName,
       sourceLine,
       sourceColumn,
     },
     pstateLinePrefix = "1234"                      consumed part of line in previous state
   }


 -- case 1: new offset is still same line
 -- e.g. offset = 7
 --        consumes ['a','b','c']
 (
   Just "1234abcde",                                full current line string
   PosState {
     pstateOffset = offset,
     pstateInput = ['d','\n','e','f','g'],          unconsumed tokens
```

```
    pstateSourcePos = SourcePos {
      sourceName = sourceName,
      sourceLine = sourceLine,                    same line
      sourceColumn = mkPos 8,                     position of first unconsumed
character in line i.e. 'd' in "1234abcd"
    },
    pstateTabWidth = ,
    pstateLinePrefix = "1234abc"                  consumed part of current line
  }
 )

 -- case 1: new offset is different line
  -- e.g. offset = 10
  --        consumes ['a','b','c','d','\n','e']
 (
   Just "efg",                                    full current line string
   PosState {
     pstateOffset = offset,
     pstateInput = ['g'],                         unconsumed tokens
     pstateSourcePos = SourcePos {
       sourceName = sourceName,
       sourceLine = mkPos$ unPos sourceLine + 1,  new line
       sourceColumn = mkPos 2,                    position of first unconsumed
character in line i.e. 'f' in "efg"
     },
     pstateTabWidth = ,
     pstateLinePrefix = "e"                       consumed part of current line
   }
 )


e.g.
1 | ABDEF                 sourceLine = 1, sourceColumn = 1, line tokens = "ABDEF",
expected token length= 1
  | ^
1 | ABDEF                 sourceLine = 1, sourceColumn = 3, line tokens = "ABDEF",
expected token length= 1
  |    ^
1 | ABDEF                 sourceLine = 1, sourceColumn = 1, line tokens = "ABDEF",
expected token length = 4
  | ^^^^
1 | ABDEFGH               sourceLine = 1, sourceColumn = 3, line tokens = "ABDEFGH",
expected token length = 4
  |    ^^^^
```

Usage: stream with **custom token** e.g. [a]
Need to implement **VisualStream** + **TraversableStream**

e.g.

```
{-# LANGUAGE FlexibleInstances #-}
{-# LANGUAGE RecordWildCards #-}

import Data.Function
import Data.Functor
import Data.List.Extra
import Data.List.NonEmpty qualified as NE
```

```
import Data.Void
import Text.Megaparsec
import Text.Megaparsec.Char

data Term = A | B | C | D | E | F | G | H | I | J | K | L | M | N |
O | P | Q | R | S | T | U | V | W | X | Y | Z | Hello | Newline
  deriving (Show, Eq, Ord)

type Parser = Parsec Void [Term]

instance VisualStream [Term] where
  showTokens _ = showTokens_ . NE.toList
  tokensLength _ cs =
    NE.toList cs
      <&> length . showTerm
      & sum

instance TraversableStream [Term] where
  reachOffset
    offset
    p@( PosState
          { pstateOffset,
            pstateInput,
            pstateSourcePos =
              SourcePos
                { sourceName,
                  sourceLine
                },
            pstateLinePrefix
          }
        ) =
        ( Just (fullCurrentLinePrefix <> showTokens_
restOfCurrentLine),
          PosState
            { pstateOffset = offset,
              pstateInput = unconsumed,
              pstateSourcePos = newPos,
              pstateTabWidth = mkPos 1,
              pstateLinePrefix = fullCurrentLinePrefix
            }
        )
      where
        -- Step 1: determine consumed and unconsumed tokens in
current state
        (consumed, unconsumed) = splitAt (offset - pstateOffset)
pstateInput

        -- Step 2: determine if current state is the same line as
the previous state
        -- If it is, then need to prepend tokens that were consumed
by the previous state
        -- in order to determine the position in the current line.
        consumedLines = split (== Newline) consumed
```

```
        isSameLine = length consumedLines == 1
        (currentLinePrefix, consumedInCurrentLine) =
          if isSameLine
            then -- case: if same line, continue from previous
state
              (pstateLinePrefix, consumed)
            else -- case: if new line, ignore previous state
              ("", last consumedLines)

        -- Step 3: compute all tokens in the current line
        fullCurrentLinePrefix = currentLinePrefix <> showTokens_
consumedInCurrentLine
        restOfCurrentLine = takeWhile (/= Newline) unconsumed

        newPos =
          SourcePos
            { sourceName = sourceName,
              sourceLine = changePos (+ (length consumedLines - 1))
sourceLine,
              sourceColumn = mkPos (length fullCurrentLinePrefix +
1)
            }

showTokens_ :: [Term] -> String
showTokens_ = intercalate "" . (showTerm <$>)

showTerm :: Term -> String
showTerm Newline = "\n"
showTerm c = show c

changePos :: (Int -> Int) -> Pos -> Pos
changePos f = mkPos . f . unPos
```

e.g. different columns

```
ghci> parseTest (single A >> single B >> single C :: Parser Term)
[X,B,C]
1:1:
  |
1 | XBC
  | ^
unexpected X
expecting A

ghci> parseTest (single A >> single B >> single C :: Parser Term)
[A,X,C]
1:2:
  |
1 | AXC
  |  ^
unexpected X
expecting B
```

```
ghci> parseTest (single A >> single B >> single C :: Parser Term)
[A,B,X]
1:3:
  |
1 | ABX
  |   ^
unexpected X
expecting C

ghci> parseTest (single A >> single B >> single C :: Parser Term)
[A,Hello ,C]
1:2:
  |
1 | AHelloC
  |  ^^^^^
unexpected Hello
expecting B
```

e.g. different lines

```
ghci> parseTest (single A >> single B >> single Newline >> single C
>> single D :: Parser Term) [A,B,Newline,X,D]           -- Newline
2:1:
  |
2 | XD
  | ^
unexpected X
expecting C
it :: ()
(0.03 secs, 1,514,168 bytes)
ghci> parseTest (single A >> single B >> single Newline >> single C
>> single D :: Parser Term) [A,B,Newline,C,X]
2:2:
  |
2 | CX
  |  ^
unexpected X
expecting D
it :: ()
```

e.g. different token lengths

```
ghci> parseTest (single A >> single B >> single C :: Parser Term)
[A,B,D,E]
1:3:
  |
1 | ABDE
  |   ^
unexpected D
expecting C
```

```
ghci> parseTest (single A :: Parser Term) [Hello,B,C]
1:1:
  |
1 | HelloBC
  | ^^^^^
unexpected Hello
expecting A

ghci> parseTest (string [A,B,C] :: Parser [Term]) [A,B,D,E]
1:1:
  |
1 | ABDE
  | ^^^
unexpected ABD
expecting ABC
```

==========

```
This
is
code
Monaco 14
```